The Journal of Open Source Software

Mesa 3: Agent-based modelling with Python in 2024

Ewout ter Hoeven [1], **Jan Kwakkel** [1], **Vincent Hess** [2], **Thomas D. Pike** [3], **Boyu Wang** [4], **and Jackie Kazil** [3]

1 Delft University of Technology (Faculty of Technology, Policy and Management), the Netherlands **2** Independent Researcher, Germany **3** George Mason University (Department of Computational Social Science), United States **4** University at Buffalo (Department of Geography), United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 December 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Mesa is an open-source Python framework for agent-based modeling (ABM) that enables researchers to create, analyze, and visualize agent-based simulations. Mesa provides a comprehensive set of tools and abstractions for modeling complex systems, with capabilities spanning from basic agent management to sophisticated representation of spaces within which agents interact. First released in 2014 and published in Masad et al. (2015) (with updates published in Kazil et al. (2020)), this paper presents Mesa in its current version (3.1.1) as of late 2024.

Statement of need

Agent-based models (ABMs), or artificial societies, are composed of autonomous heterogeneous agents interacting locally with other agents. These interactions give rise to emergent phenomena. The aggregate dynamics of a system under study emerge from these local interactions (Joshua M. Epstein & Axtell, 1996) (Joshua M. Epstein, 1999). This type of modeling quickly grew more sophisticated, requiring frameworks to execute them. This led to the establishment of NetLogo and MASON.

However, before Mesa, there was no modern Python-based framework for ABMs that integrated with the scientific Python ecosystem. Since it's creation in 2014, Mesa has been applied to modeling everything from economics and sociology to ecology and epidemiology and has been cited in more than 500 papers and 800 authors. Today, Mesa advanced usability and stabilized functionality with its most recent major release (3+). These features include enhanced management of agents, data collection advancements, improved visualization framework, and making it easier for researchers to create and analyze complex simulations.

Core capabilities

Mesa is a Python-based framework for ABM that provides a comprehensive set of tools for creating, running, and analyzing ABMs. Mesa integrates with the wider scientific Python ecosystem with libraries such as NumPy, pandas, Matplotlib, NetworkX, and more. Mesa is implemented in pure Python (3.11+) with a modular architecture separating: 1. Core ABM components (*i.e.,* agents, spaces, agent activation, control over random numbers) 2. Data collection and support for model experimentation 3. Visualization systems

This design allows selective use of components while enabling extension and customization.

Core ABM components

Model

The central class in Mesa is the Model. The user extends the model. Within it, the user instantiates the space and populates it with agent instances. Since ABMs are typically stochastic simulations, the model also controls the random number generation.

Agents

Central to ABMs are the autonomous heterogeneous agents. Mesa provides a variety of base agent classes which the user can subclass. In its most basic implementation, such an agent subclass specifies the `__init__` and `step` method. Any subclass of the basic mesa agent subclass registers itself with the specified model instance, and via `agent.remove` it will remove itself from the model. It is strongly encouraged to rely on `remove`, and even extend it if needed to ensure agents are fully removed from the simulation.

Agent management

One significant advancement of Mesa 3+ is expanded functionality around agent management. The new `AgentSet` class provides methods that allow users to filter, group, and analyze agents, making it easier to express complex model logic.

```
# Select wealthy agents and calculate average wealth
wealthy = model.agents.select(lambda a: a.wealth > 1000)
avg_wealth = wealthy.agg("wealth", func=np.mean)

# Group agents by type and apply behaviors
grouped = model.agents.groupby("species")
for species, agents in grouped:
    agents.shuffle_do("reproduce")
```

Spaces

Mesa 3 provides both discrete (cell-based) and continuous space implementations. In discrete spaces, an agent occupies a cell. Mesa implements discrete spaces using a doubly-linked structure where each cell maintains connections to its neighbors. The framework includes several discrete space variants with a consistent API:

- Grid-based: `OrthogonalMooreGrid`, `OrthogonalVonNeumanGrid`, and `HexGrid`
- Network-based: `Network` for graph-based topologies
- Voronoi-based: `VoronoiMesh` for irregular tessellations

Example grid creation:

```
grid = OrthogonalVonNeumannGrid(
    (width, height), torus=False, random=model.random
)
```

Mesa provides specialized agent classes for spatial interactions in the discrete spaces:

- `FixedAgent`: Is assigned to a cell, can access this cell, but cannot move to another cell.
- `CellAgent`: Can move between cells
- `Grid2DMovingAgent`: Extends `CellAgent` with directional movement methods

All discrete spaces support PropertyLayers - efficient numpy-based arrays for storing cell-level properties:

```
grid.create_property_layer("elevation", default_value=10)
high_ground = grid.elevation.select_cells(lambda x: x > 50)
```

DRAFT

For models where agents need to move continuously through space rather than between discrete locations, `ContinuousSpace` allows agents to occupy any coordinate within defined boundaries:

```
space = ContinuousSpace(x_max, y_max, torus=True)
space.move_agent(agent, (new_x, new_y))
```

Time advancement

Typically, ABMs rely on incremental time progression or ticks. For each tick, the step method of the model is called. This activates the agents in some way. The most frequently implemented approach is shown below, which runs a model for 100 ticks.

```
model = Model(seed=42)

for _ in range(100):
    model.step()
```

Generally, within the step method of the model, one activates all the agents. The newly added `AgentSet` class provides a more flexible way to activate agents replacing the fixed patterns previously available.

```
model.agents.do("step")  # Sequential activation

model.agents.shuffle_do("step")  # Random activation

# Multi-stage activation:
for stage in ["move", "eat", "reproduce"]:
    model.agents.do(stage)

# Activation by agent subclass:
for klass in model.agent_types:
    model.agents_by_type[klass].do("step")
```

Mesa also supports next-event time progression. In this approach, the simulation consists of time-stamped events that are executed chronologically, with the simulation clock advancing to each event's timestamp. This enables both pure discrete event-based models and hybrid approaches combining incremental time progression with event scheduling on the ticks as shown below.. This latter hybrid approach allows combining traditional ABM time steps with the flexibility and potential performance benefits of event scheduling.

```
# Pure event-based scheduling
simulator = DiscreteEventSimulator()
model = Model(seed=42, simulator=simulator)
simulator.schedule_event_relative(some_function, 3.1415)

# Hybrid time-step and event scheduling
model = Model(seed=42, simulator=ABMSimulator())
model.simulator.schedule_event_next_tick(some_function)
```

Visualization

Mesa's visualization system, SolaraViz, provides interactive browser-based model exploration:

```
visualization = SolaraViz(
    model,
    [
        make_space_component(agent_portrayal),
        make_plot_component(["population", "average_wealth"]),
        lambda m: f"Step {m.steps}: {len(m.agents)} agents"
```

DRAFT

```
    ],
    model_params=parameter_controls
)
```

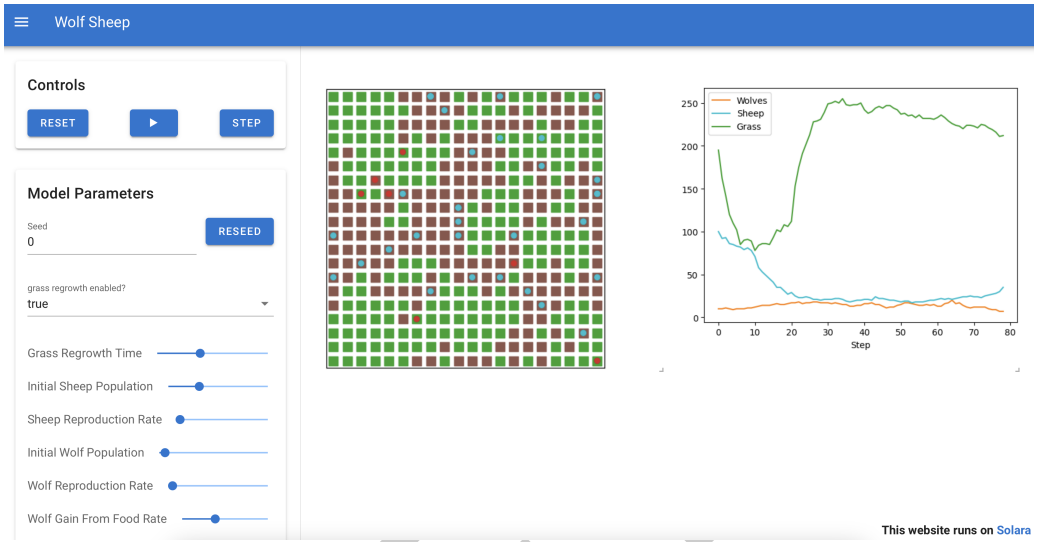

Figure 1: A screenshot of the WolfSheep Model in Mesa

Key features include: - Interactive model controls - Real-time data visualization - Customizable agent and space portrayal - Support for multiple visualization types including grids, networks, and charts

Experimentation and analysis

Data collection

Mesa's DataCollector enables systematic data gathering during simulations:

```
collector = DataCollector(
    model_reporters={"population": lambda m: len(m.agents)},
    agent_reporters={"wealth": "wealth"},
    agenttype_reporters={
        Predator: {"kills": "kills_count"},
        Prey: {"distance_fled": "flight_distance"}
    }
)
```

The collected data integrates seamlessly with pandas for analysis:

```
model_data = collector.get_model_vars_dataframe()
agent_data = collector.get_agent_vars_dataframe()
```

Parameter sweeps

Mesa supports systematic parameter exploration:

```
parameters = {
    "num_agents": range(10, 100, 10),
    "growth_rate": [0.1, 0.2, 0.3]
}
results = mesa.batch_run(
```

DRAFT

```
    MyModel,
    parameters,
    iterations=5,
    max_steps=100
)
```

Applications

Mesa has been applied across diverse domains, including:

- Sustainability and food security modeling, including supply chain optimization and resource allocation (Namany et al., 2020)
- Infrastructure resilience and post-disaster recovery planning (Sun & Zhang, 2020)
- Building stock modeling for energy demand and emissions analysis (Nägeli et al., 2020)
- Market modeling, including renewable energy auctions and consumer behavior (Anatolitis & Welisch, 2017)
- Transportation optimization, such as combined truck-drone delivery routing (Leon-Blanco et al., 2022)
- Recommender systems analysis examining consumer-business value tradeoffs over time (Ghanem et al., 2022)
- Climate adaptation modeling examining household-level behavioral responses to environmental shocks (Taberna et al., 2023)

These applications showcase Mesa's versatility in modeling complex systems with autonomous interacting agents, whether representing individual consumers, infrastructure components, buildings, or vehicles.

The framework is particularly suited for: - Models with heterogeneous agent populations - Systems requiring sophisticated spatial interactions - Interactive exploration of parameter spaces - Teaching and learning agent-based modeling

Community and ecosystem

Mesa has grown into a complete ecosystem with extensions including: - Mesa-Geo for geospatial modeling [wang2022mesa] - Mesa-Frames for high-performance simulations - A rich collection of community-contributed extensions, example models, and tutorials

Acknowledgements

The advancements leading to Mesa 3 were developed by six maintainers (the authors) and an active community with over 140 contributors. We would especially like to thank David Masad for his foundational work on Mesa and rht for his work as maintainer in 2022 to 2024.

Conclusions

Mesa 3 introduces significant advancements to the Python ABM framework, enhancing the core toolkit with greater control, interactivity, and speed for researchers. These notable improvements, paired with its foundational integration with the scientific Python ecosystem, modular architecture, and active community, make it an indispensable tool for researchers across disciplines working in Python who need to create and analyze agent-based models.

DRAFT

References

Anatolitis, V., & Welisch, M. (2017). Putting renewable energy auctions into action–an agent-based model of onshore wind power auctions in germany. *Energy Policy*, *110*, 394–402. https://doi.org/10.1016/j.enpol.2017.08.024

Epstein, Joshua M. (1999). Agent-based computational models and generative social science. *Complexity*, *4*(5), 41–60.

Epstein, Joshua M., & Axtell, R. (1996). *Growing artificial societies: Social science from the bottom up*. Brookings Institution Press; MIT Press. ISBN: 9780262550253

Ghanem, N., Leitner, S., & Jannach, D. (2022). Balancing consumer and business value of recommender systems: A simulation-based analysis. *Electronic Commerce Research and Applications*, *55*, 101195. https://doi.org/10.1016/j.elerap.2022.101195

Kazil, J., Masad, D., & Crooks, A. (2020). Utilizing python for agent-based modeling: The mesa framework. *Social, Cultural, and Behavioral Modeling: 13th International Conference, SBP-BRiMS 2020*, 308–317. https://doi.org/10.1007/978-3-030-61255-9_30

Leon-Blanco, J. M., Gonzalez-R, P. L., Andrade-Pineda, J. L., Canca, D., & Calle, M. (2022). A multi-agent approach to the truck multi-drone routing problem. *Expert Systems with Applications*, *195*, 116604. https://doi.org/10.1016/j.eswa.2022.116604

Masad, D., Kazil, J. L., & others. (2015). Mesa: An agent-based modeling framework. *SciPy*, 51–58. https://doi.org/10.25080/Majora-7b98e3ed-009

Nägeli, C., Jakob, M., Catenazzi, G., & Ostermeyer, Y. (2020). Towards agent-based building stock modeling: Bottom-up modeling of long-term stock dynamics affecting the energy and climate impact of building stocks. *Energy and Buildings*, *211*, 109763. https://doi.org/10.1016/j.enbuild.2020.109763

Namany, S., Govindan, R., Alfagih, L., McKay, G., & Al-Ansari, T. (2020). Sustainable food security decision-making: An agent-based modelling approach. *Journal of Cleaner Production*, *255*, 120296. https://doi.org/10.1016/j.jclepro.2020.120296

Sun, J., & Zhang, Z. (2020). A post-disaster resource allocation framework for improving resilience of interdependent infrastructure networks. *Transportation Research Part D: Transport and Environment*, *85*, 102455. https://doi.org/10.1016/j.trd.2020.102455

Taberna, A., Filatova, T., Hadjimichael, A., & Noll, B. (2023). Uncertainty in boundedly rational household adaptation to environmental shocks. *Proceedings of the National Academy of Sciences*, *120*(44), e2215675120. https://doi.org/10.1073/pnas.2215675120

DRAFT